ChatGPT

本地 Agent/LLM 消息历史压缩技术与工程实践研究报告

Executive Summary

截至 **2026 年 8 月 12 日**，主流 Agent 框架对长历史的处理已经形成一个相当一致的工程共识：**不要把完整历史无限重放给模型；应把“完整事实记录”和“本轮模型可见工作集”分离，并在每次 model call 前进行预算化裁剪。** LangChain 提供 trim/delete/summarize，AutoGen 同时提供 recent-N、token-limit 与 head+tail context，LlamaIndex 以 token limit + FIFO/long-term memory 管理历史，Semantic Kernel 有独立 ChatHistoryReducer，Anthropic 甚至把“清理旧 tool result”做成了 API 原语。 [1]

针对你当前项目，**最值得采用的不是简单 `recent 6 messages`，而是一个无额外 LLM 调用的 hybrid rule-based compactor：`first task + pinned state + atomic tool exchanges + recent window + token budget`**。关键原则是：**按 tool exchange 原子组裁剪，而不是按单条 message 裁剪**；否则很容易产生 `tool_use` / `tool_result` 孤儿消息。LangChain 官方明确警告多数 provider 要求 tool call 后存在对应 tool result，Anthropic 对 thinking + tool use 还有更严格的完整性要求。 [2]

对你现在的 `LLMRunExecutor`，短期最佳落点是在 **每次 provider call 之前生成 `CompactionPlan`，只构造一个短的 request view，不销毁原始 history**。`100k` cumulative run budget 继续保留；新增/完善的是 **per-request history budget** 与 preflight token check。这样同时获得低侵入、可回滚、可审计和 deterministic tests。

长期来看，应把 active conversation 视为类似操作系统的 working set，把完整事件、artifact、工具输出留在外部可恢复存储，只把当前任务真正需要的状态重新装入上下文。这个方向与 MemGPT 的 virtual context management、LangGraph checkpointing、LlamaIndex 的 short-/long-term memory 以及最新主流平台的 compaction 设计高度一致。 [3]

背景与目标

你的当前问题已经可以精确拆成两层。

第一层 **Defect A / static context selection** 已经基本解决：

```
Task
  ↓
ContextSelector
  ↓
ContextSnapshot
  ↓
SizeEstimator
  ↓
ContextTrimmer
```

```
  ↓
RunContextBuilder
```

第二层才是现在仍使真实 smoke test 失败的核心：

```
Round 1:
static context + task

Round 2:
static context + task + assistant + tool result

Round 3:
static context + task + assistant + tool result + assistant + tool result

...

累计 API usage > run-level 100k
```

这并不是某一家 provider 特有的问题。Anthropic 的文档明确说明，每次请求的 context 都包括 system、所有 messages、tool result、文档和 tool definitions；历史会逐轮积累，而且 prompt caching 即使降低价格，也**不会把这些 token 从 context window 中移除**。Claude Code 的成本文档也直接指出，长会话中的每次 tool request 都会再次携带历史。[4]

LangChain 官方把解决办法归为四类：trim、delete、summarize 和 custom strategy，并特别推荐在 model call 前执行 trimming；AutoGen 则把这件事抽象成 `model_context`，既有只保留最近 N 条的 `BufferedChatCompletionContext`，也有 token 限制版本与 head-and-tail 版本。[5]

LlamaIndex 当前 `Memory` 的默认短期记忆也是“最后 X 条且满足 token limit”，超出短期预算后可 flush 至长期 memory；其 memory block 还能用 priority 控制哪些长期内容优先留下。[6]

Semantic Kernel 则明确把 reduction 做成接口 `IChatHistoryReducer`，内置 truncation 与 summarization，并要求 reducer 保留 system message。[7]

这些实现背后的核心并不是“模型窗口不够大”。研究已经表明，即使模型支持很长 context，相关信息位于长上下文中部时，利用效果也可能下降；更多 context 不等于更可靠。`Lost in the Middle` 的实验发现模型通常在相关信息位于开头或结尾时表现更好。Anthropic 现在的官方文档也直接称这种长上下文性能退化为 context rot，并强调应主动 curate context。[8]

因此，本项目的目标不应该是：

```
让 messages 尽可能多地塞进模型。
```

而应该是：

```
完整历史 = durable system of record

        ≠
```

```
本轮 LLM context = bounded working set
```

建议最终形成四层预算概念：

```
Static context budget
        │
        ├─── 当前已有 DEFAULT_CONTEXT_BUDGET = 12000
        │
History request budget
        │
        ├─── 每次调用允许进入模型的动态历史
        │
Provider context-window budget
        │
        ├─── system + tools + static + history + output reserve
        │
Run cumulative budget
        │
        └─── 当前 100k 成本/失控保护，继续保留
```

特别需要避免把最后两者混为一谈。OpenAI 的 reasoning 模型文档也说明隐藏 reasoning tokens 属于 output usage 并占用 context/billing；Anthropic 则把 cache read/create/input 分开报告，但这些部分仍占 context window。因此未来 usage accounting 最好至少区分 **request logical context、provider-reported usage 与 run cumulative cost**。[9]

方案清单与比较表

下面的评分以你的目标——**本地 Agent runtime、低侵入、无额外 LLM 优先、工具循环安全**——为准，而不是通用聊天产品的绝对排名。

方案名	需额外 LLM	可解释性	实现复杂度	风险等级	适用场景	优先级建议
Recent-N 消息保留	否	高	低	中	普通多轮聊天、短 tool loop	中
System/First + Recent / Head-Tail	否	高	低	低～中	Agent task、固定初始目标	**高**
Tool-exchange Sliding Window	否	高	低～中	**低**	多轮工具调用	**最高**
Token-budgeted Trimming	否	高	中	低	所有长上下文场景	**最高**
Message/Exchange Heuristic Scoring	否	高	中	中	工具类型混杂、关键结果稀疏	高

方案名	需额外 LLM	可解释性	实现复杂度	风险等级	适用场景	优先级建议
Tool-result Pinning / Selective Clearing	否	高	中	低	文件读取、测试、搜索大量返回	**最高**
Delta-Encoding / Tool Output Normalization	否	高	中	中	重复 state、日志、diff、大结果	高
Checkpointing + Snapshotting	否	很高	中	**低**	长时间任务、恢复、审计	**最高/基础设施**
Chunking + Summarization	**是**	中	中～高	中～高	超长会话、跨阶段任务	中/长期
Embedding-based Relevance Retrieval	否或 embedding	中	高	中	跨很长历史的信息召回	中
Learned Prompt Compression / LLMLingua 类	本地小模型通常需要	低～中	高	高	极端长 prompt、研究型优化	低

这里的前三类并非理论设想。AutoGen 官方目前就提供 last-N `BufferedChatCompletionContext`、token-limit context 与 first-N/last-M `HeadAndTailChatCompletionContext`；LangChain 官方文档展示的 trimming 示例也是保留第一条和最近几条消息。[10]

Token-budgeted history 同样已经是主流框架共同选择：LangChain 支持按 token trim；AutoGen 有 `TokenLimitedChatCompletionContext`；LlamaIndex 的 `Memory` 原生以 token limit 管理短期历史；Semantic Kernel 将 token-based reduction 列为标准策略之一。[11]

对 tool-heavy agent，Anthropic 的 context editing 尤其值得参考：官方 `clear_tool_uses` 会在超过 threshold 后**按时间从旧到新清除 tool results**，默认保留最近三个 tool-use/result pair，还支持 `exclude_tools` 永久保护特定工具，并默认保留 tool input。也就是说，“旧 tool output 往往是最佳压缩对象，但要允许 pin 某些工具”已经是生产 API 层面的模式。[12]

Checkpoint + active working set 则对应 LangGraph 的 checkpointer：完整 graph state 可持久化为 checkpoints，用于恢复、HITL、time travel 和 fault tolerance，而不是要求所有旧状态始终出现在下一次 model prompt 中。[13]

每方案详细说明

Recent-N 保留

描述：只保留最后 N 条 `LLMMessage`。这是 AutoGen `BufferedChatCompletionContext` 的直接模式，也是所有策略里实现最简单的一种。[14]

优点是 O(n) 或甚至 O(1) 的实现、完全 deterministic、无额外模型调用、极易测试。缺点也明显：**message 不是安全的压缩单位**。一条 assistant message 可能包含 `tool_use`，下一条 user message 才携带 `tool_result`；在边界中间切开就会得到无效 protocol history。LangChain 官方特别警告 tool-call assistant message 应跟着对应 result。[15]

本项目不建议直接采用：

```
retained_indices = list(
    range(max(0, len(messages) - 6), len(messages))
)
```

而应只把它作为 fallback 或 unit test baseline。

测试必须覆盖：

```
tool_use | CUT | tool_result
```

绝不能成为合法输出。

优先级：中。推荐：不单独使用。

System/First + Recent，也就是 Head-Tail

模式是：

```
[pinned head] + [recent tail]
```

AutoGen 已正式提供 `HeadAndTailChatCompletionContext`；LangChain 的官方 trimming 示例同样拿 `first_msg` 再拼接近期消息。 10

对你项目尤其合适，因为初始 user task：

```
"implement a login feature"
```

是整个 agent run 的目标锚点。一旦只做 recent-N，经过大量 tool loop 后初始目标可能消失。

建议规则：

```
PIN:
    initial task message

KEEP:
    most recent complete interaction groups
```

如果 system prompt 在你 provider API 中并不属于 `messages`，则 system 根本不进入 compactor；若未来统一成 message，则 system 必须永久 pin。Semantic Kernel 的 reducer 也明确保留 system message。 7

测试重点：

```
100 messages 后:
initial task 仍存在
latest tool exchange 完整存在
old middle 被移除
```

优先级：高。推荐：采用，但与 token budget + tool grouping 组合。

Tool-exchange Sliding Window

这是我认为你项目短期最重要的改动。

不要把：

```
message
```

当压缩原子，而要把：

```
assistant(tool_use)
        +
user(tool_result)
```

当成：

```
ToolExchange
```

在 Claude 体系下，如果启用了 thinking，工具交互还可能包含必须一起返回的 thinking block；Anthropic 明确要求在提交对应 tool result 时，tool request 附带的 thinking block必须保持完整且不可修改，否则 API 会拒绝请求。 16

因此逻辑应类似：

```python
@dataclass(frozen=True)
class MessageGroup:
    indices: tuple[int, ...]
    kind: Literal[
        "task",
        "dialogue",
        "tool_exchange",
    ]
```

然后：

```
groups = group_history(messages)

keep(
    first_task,
    last_k_complete_tool_exchanges,
    latest_non_tool_messages,
)
```

这和 Semantic Kernel 的实践经验也一致：其 `threshold_count` 专门用于避免 truncation 后产生孤立消息，而 SK 过去还实际出现过“truncation 切掉 tool-call，留下孤儿 tool message”的 bug，说明这不是理论风险。[17]

测试至少包括：

```
assistant(tool_use A)
user(tool_result A)

assistant(tool_use B)
user(tool_result B)
```

任何 `CompactionPlan` 都只能：

```
两条一起留
或
两条一起删
```

优先级：最高。推荐：立即采用。

Token-budgeted Trimming

message count 很容易被一个巨大 tool output 击穿：

```
1 个 message
=
20k tokens
```

所以 N-message window 不能作为最终预算机制。LangChain、AutoGen、LlamaIndex、Semantic Kernel 都把 token limit 作为独立策略。[11]

建议你的 `HistoryCompactor` 不接收固定 `RECENT_MESSAGE_COUNT=6` 作为主控制，而是最终接收：

```
@dataclass(frozen=True)
class HistoryBudget:
    max_tokens: int
```

或者复用一个更通用的 budget protocol。

核心算法：

```
groups = group_history(messages)

retained = mandatory_groups()

remaining = budget.max_tokens - estimate(retained)

for group in reversed(optional_groups):
    size = estimator.estimate(group)

    if size <= remaining:
        retained.add(group)
        remaining -= size
```

注意这里和你之前的 static `ContextTrimmer` 有一点不同：历史的每个 tool exchange 是相对独立的，因此**可以讨论是否 skip oversized old group 后继续寻找更近/更小 group**；但短期为了 deterministic 和时间连续性，我仍建议 newest→oldest 连续装填，达到边界即停。

provider 有可靠 token counter 时应优先使用真实 counter。Anthropic 提供 token counting API；AutoGen 的 token-limited context要求 model client 实现 `count_tokens()`；LlamaIndex 则允许自定义 tokenizer，默认实现亦进行真实 token counting。[18]

你的当前 `len(str(message)) // 4` 可以继续作为 provider-independent fallback，但必须明确叫 **estimate**，不能用于宣称“绝不会超过 provider context window”。

优先级：最高。推荐：立即采用。

Message / Exchange Heuristic Scoring

它不是替代 sliding window，而是在 mandatory pins 之后决定：

> 哪些较旧信息值得留下？

可用纯规则评分：

```
score =
    recency_weight
    + tool_error_weight
    + side_effect_weight
```

```
    + user_constraint_weight
    + test_result_weight
    + current_task_keyword_weight
```

例如：

```
read_file 旧结果                score 低
search old result               score 低
write_file success              score 高
pytest FAILED                   score 高
git diff                        score 高
human approval                  永久 pin
```

LlamaIndex 的 memory block 已经使用 priority 来决定超预算时谁先被移除；LongLLMLingua 则进一步证明“根据当前问题识别关键 context 再分配压缩预算”能够比盲目保留长上下文更有效。 19

但风险是：

```
score heuristic 错
→
旧的关键事实被删除
```

所以必须有不可覆盖的 mandatory layer：

```
mandatory pin
    >
heuristic score
```

建议初版接口：

```python
class MessageImportancePolicy(Protocol):
    def score(
        self,
        group: MessageGroup,
    ) -> int:
        ...
```

测试应使用固定 feature，不允许时间或随机因素导致 nondeterministic output。

优先级：高，但在 atomic grouping + token budget 之后。

Tool-result Pinning / Selective Clearing

这是最贴近你当前 Agent 的方案。

Anthropic 的 context editing 明确认为旧 file contents/search results 在 agent 已经处理之后通常不必永久保留；API 默认从最旧 tool results 开始清理、保留最近 tool uses，并可通过 `exclude_tools` 保护不能清除的工具。[12]

这可以直接转化成本地规则：

```
PIN_TOOL_RESULTS = {
    "write_file",
    "apply_patch",
    "git_commit",
    "run_tests",
}

EVICTABLE_TOOL_RESULTS = {
    "read_file",
    "list_directory",
    "search",
}
```

实际更安全的是根据**语义类别**而非名字：

```
read-only / reproducible
    → 容易清除

state-changing
    → pin

verification
    → 至少保留最后结果

error
    → 保留直到成功 recovery
```

例如：

```
read_file(foo.py)
```

旧内容通常可以重新读取；

但：

```
apply_patch(... success)
```

说明 agent 已改变外部状态，不能简单忘记。

Anthropic 的 API 在清除时会留下 placeholder 告诉模型该结果被移除了，这也是值得借鉴的设计。[12]

你的 `CompactionPlan` 当前只能 retain/drop，因此第一版可以整组删除；第二版可扩展成：

```
replacement_messages
```

或：

```
CompactionAction(
    indices=(...),
    replacement="[older read_file result omitted; artifact remains retrievable]",
)
```

但不要现在污染已经完成的 `CompactionPlan`，可以后续单独升级。

优先级：最高。推荐采用。

Delta-Encoding / Tool Output Normalization

这是一个工程模式，不是目前主流框架统一命名的“history compaction API”。

核心思想：

如果工具每轮都返回：

```
完整 5,000 行状态
```

不应该先把 5,000 行塞进 history，再想办法压缩；最好源头就返回：

```
change since previous state
```

例如：

```
之前：
pytest 输出 8k

之后：
pytest:
417 passed
2 failed
FAIL:
tests/foo.py::test_x
tests/bar.py::test_y
```

或 Git 工作树：

```
base snapshot id = abc123

delta:
M src/a.py
A tests/test_a.py
```

Semantic Kernel 的 function-calling 文档明确建议 tool return 尽量精简，只返回模型需要的信息，因为函数结果还会再次被模型消费并产生 token 成本。Anthropic 的 programmatic tool calling 更进一步：多个中间工具结果可以在 code execution 中处理，**中间结果不进入 Claude context，只把最终结果给模型**。 20

安全条件是必须有可重建 baseline：

```
snapshot S0
+
delta D1
+
delta D2
=
state S2
```

测试：

```
assert reconstruct(snapshot, deltas) == full_state
```

对于日志则不一定需要完全可逆，但原始 artifact 必须持久化。

优先级：高。特别适合 coding agent，但属于 tool layer 优化，不应塞进 HistoryCompactor 本身。

Checkpointing + Snapshotting

这是最容易被误解的一点：

> 压缩 active LLM history，不代表删除系统历史。

LangGraph 的 checkpointer 会在执行步骤保存 state snapshot，用于恢复、HITL、time travel 和 fault tolerance；Microsoft 的 Agent Framework/架构指导也强调长任务应把 progress、intermediate results、conversation state 持久化到 durable store，而不是只依赖 model context。 21

你的项目已经存在 `events.jsonl` 这一类执行记录，因此架构上非常适合：

```
Full immutable history
        ↓
events / artifacts / snapshots
        |
        | derive
```

```
        ▼
Active compacted LLM view
```

这解决了最重要的安全问题：

```
Compactor 误删
≠
数据永久丢失
```

未来还可以：

```
model: "I need the old test result"

runtime:
    load artifact/checkpoint
    inject relevant portion
```

这就是 MemGPT 所描述的 virtual context management 思路：上下文窗口是快速 working memory，外部存储承担更大的长期信息层。 22

优先级：最高作为基础设施。推荐采用，而且不要等到 LLM summarization 才做。

Chunking + Summarization

这是主流平台最终都会提供的方案，但不符合你当前“无额外 LLM 调用优先”的短期要求。

LangChain `SummarizationMiddleware` 可以在 token threshold 触发时，用模型总结旧 history 并保留近期消息；Semantic Kernel 有 `ChatHistorySummarizationReducer`；LlamaIndex 的旧 `ChatSummaryMemoryBuffer` 也是相同模式；Anthropic client/server compaction 和 OpenAI `/responses/compact` 都属于 provider 层的 summary-based context reduction。 23

Anthropic SDK compaction 的具体流程很典型：

```
threshold
 ↓
inject summary request
 ↓
Claude creates summary
 ↓
full history replaced by summary
 ↓
continue
```

官方示例可把接近 100k token 的历史压到约几千 token，但该方式本质上增加了一次 model inference。 12

主要风险：

```
summary hallucination
summary omission
错误把旧假设写成事实
丢失精确路径/ID/error text
```

OpenAI 当前 standalone compaction 更进一步返回 opaque/encrypted compaction item，不设计给人解释，因此 provider compaction虽然方便，却不符合你现在特别重视的可解释 `CompactionPlan` 模式。 [24]

长期如果采用，建议 summary 使用结构化 schema：

```
Task
Decisions
Completed actions
Artifacts
Observed facts
Failed attempts
Open questions
Current state
Next actions
```

并始终保留原始 events/artifacts。

优先级：中/长期。短期不采用。

Embedding-based Relevance Ranking

思路：

```
旧 history
 ↓
chunks / exchanges
 ↓
embeddings
 ↓
retrieve top-k relevant to current task/state
```

AutoGen 当前支持 vector DB memory，LlamaIndex 有 `VectorMemoryBlock`，LangGraph/Store 也支持 semantic search。 [25]

它适合：

```
超长运行
跨很远 turn 回忆旧事实
长期项目记忆
```

但不适合单独替代当前短期 tool loop。

LlamaIndex 的官方文档甚至明确指出 VectorMemory **不保证消息原有顺序**，而只是返回与最近用户消息最相似的内容。对于 tool protocol：

```
tool_use
tool_result
```

顺序和配对是强约束，所以 semantic retrieval 必须在**已经形成的 exchange/chunk 上**运行，不能对单条 raw messages 独立检索。 6

如果用本地 SentenceTransformer，可以没有在线 LLM 成本，但仍增加：

- embedding runtime
- index persistence
- model dependency
- cache invalidation
- relevance debugging

优先级：中。适合作为中期 recall layer，不适合作为短期安全裁剪器。

Learned Prompt Compression / LLMLingua 类

LLMLingua、LongLLMLingua 等研究通过小模型或 token-level relevance 对 prompt 删除低信息 token；LongLLMLingua还做 query-aware compression 和重排序。论文报告在一些 benchmark 上获得很高压缩率并降低成本/延迟。 26

这类方法的主要问题不是能力，而是与你当前 runtime 的目标不匹配：

```
输入：
structured LLMMessage
tool_use
JSON
paths
IDs
error logs

↓ token-level compressor

可能输出：
结构损坏
```

```
JSON 损坏
call_id 消失
```

它更适合：

```
documents
RAG passages
natural-language prompt
```

而不是第一阶段直接处理 tool protocol history。

如果未来使用，应只压缩：

```
tool result textual payload
```

而绝不压缩 envelope：

```
role
type
tool_use_id
tool name
JSON control structure
```

优先级：低，属于长期 R&D。

三条实施路线

短期路线：Deterministic Tool-Aware Budgeted Compactor

这是我对你项目的明确推荐，**完全不新增 LLM 调用**。

不要直接继续此前那个：

```
RECENT_MESSAGE_COUNT = 6
```

版本作为最终 4B-2。

应把 4B-2 稍微升级为：

```
LLMMessage[]
    ↓
MessageGroupBuilder
    ↓
RuleBasedHistoryCompactor
```

```
    ↓
CompactionPlan(retained_indices)
    ↓
HistoryCompactionExecutor
    ↓
request_messages
```

具体文件建议：

```
src/context/history/

history_analysis.py                ✅
history_analyzer.py                ✅
rule_based_history_analyzer.py     ✅

compaction_plan.py                 ✅

message_group.py                   NEW
message_group_builder.py           NEW

history_budget.py                  NEW
history_compactor.py               NEW
rule_based_history_compactor.py    NEW

compaction_executor.py             NEW
```

第一版 group 分类不用复杂：

```
@dataclass(frozen=True)
class MessageGroup:
    indices: tuple[int, ...]
    kind: str
```

识别：

```
initial task
normal interaction
assistant tool_use + user tool_result
```

Compactor 规则顺序：

```
mandatory:
    initial task
    latest message
    incomplete/current tool exchange
    protected tool exchanges
```

```
then:
    recent complete tool exchanges

until:
    history token budget reached
```

推荐接口：

```python
class HistoryCompactor(Protocol):
    def compact(
        self,
        messages: list[LLMMessage],
        budget: HistoryBudget,
    ) -> CompactionPlan:
        ...
```

执行层保持简单：

```python
def apply_compaction_plan(
    messages: list[LLMMessage],
    plan: CompactionPlan,
) -> list[LLMMessage]:
    return [
        messages[index]
        for index in plan.retained_indices
    ]
```

然后 **5D-6B-4C** 只在 `LLMRunExecutor` 的 provider call 前：

```
full_messages.append(...)

plan = self._history_compactor.compact(
    full_messages,
    self._history_budget,
)

request_messages = apply_compaction_plan(
    full_messages,
    plan,
)

response = provider.complete(
    messages=request_messages,
)
```

关键是：

```
full_messages 不改
```

因此：

```
audit/recovery history
           ≠
request view
```

这种 before-model reduction 正是 LangChain 推荐的集成点；Anthropic context editing 也采用“客户端保存完整历史、服务端只对本轮 model view 做 editing”的设计，因此不用让 durable state 跟着被压缩。[27]

短期再加一个 preflight：

```
remaining_run_budget = (
    max_total_tokens
    - total_input_tokens
    - total_output_tokens
)

if estimated_request_tokens + output_reserve > remaining_run_budget:
    compact_more_or_fail_before_provider_call()
```

不要删除 cumulative 100k。

而是：

```
per-request history budget
        ↓
降低每轮 usage

run cumulative 100k
        ↓
继续防止 runaway loop
```

回归测试至少覆盖：

```
initial task 永不丢
tool_use/result 永不拆
当前未完成 tool exchange 永不删
最近 K 组保留
old read-only results 可删
protected tool results 保留
under-budget 完全不变
原 messages 完全不修改
CompactionPlan deterministic
```

```
budget=0 仍保留 mandatory state
重复 compact 得到相同结果
provider call 收到 compacted copy
事件日志仍能看到 full history
```

建议验收指标不是“希望感觉变好了”，而是：

```
历史超过阈值的请求：
history_tokens_after / history_tokens_before <= 0.50

provider request:
100% <= configured request history budget

tool protocol validation:
100% valid

20 个固定 tool-heavy smoke runs:
Maximum total token limit exceeded = 0

与未压缩 baseline 相比：
task success rate 下降 <= 5 percentage points
```

其中 `50%` 和 `20 runs` 是**工程验收目标**，不是文献保证值。

预计侵入性：低。

真正修改 runtime 的地方应主要集中在：

```
LLMRunExecutor provider call boundary
```

而 `RunContextBuilder` 不需要再改变。

中期路线：Rules + Durable Checkpoints + Local Semantic Recall

短期稳定后，不要马上跳 online summary。

加入：

```
Full events
   ↓
checkpoint / artifact store

Active context
   ↓
rule compactor
```

```
Need old information?
   ↓
local retrieval
```

文件可以演进：

```
src/context/history/
    checkpoint_store.py
    history_snapshot.py
    history_retriever.py
    local_embedding_history_retriever.py

src/context/
    artifacts/
        artifact_store.py
```

这一层优先把“大结果”外置：

```
read_file 20k
    ↓
artifact://run/X/tool/Y

history:
"read_file result stored as artifact Y; relevant files: ..."
```

然后需要时重新 retrieve。

LangGraph 的 checkpoint/store 分离、LlamaIndex 的 short-term + long-term memory、AutoGen vector memory 都支持这种“active state 与外部 recall 分离”的体系。[28]

Embedding 可以是：

```
local sentence-transformer
```

因此不产生在线 LLM 请求。

但是 retrieval 单元应该是：

```
MessageGroup / artifact / checkpoint
```

而不是 raw individual message。

验收增加：

```
同一 query top-k deterministic（固定模型版本）
tool exchange 永不被拆成 embedding item
retrieved state 带 artifact ID
被 compact 的内容能从 checkpoint 恢复
重启进程后仍可恢复
```

目标指标：

```
p95 request history tokens
比无 compaction baseline 至少下降 50%

跨 30+ tool interactions:
token-limit failure = 0

old-state recall benchmark:
>= 90% 的预定义关键事实可正确恢复

checkpoint reconstruction:
100%
```

最大风险是 semantic retrieval 的 false negative。

缓解办法：

```
semantic retrieval
+
exact artifact lookup
+
recent window
+
mandatory pins
```

绝不能让 embedding 成为唯一信息来源。

长期路线：Hierarchical Memory + Optional Summarization + Semantic Ranking

长期可以形成：

```
Tier 0
System / Task / Constraints

Tier 1
Current working set

Tier 2
```

```
Recent exchanges

Tier 3
Structured summaries/checkpoints

Tier 4
Raw artifacts/events
```

这与 MemGPT 的 virtual memory 类比非常接近：有限的 active context 类似主存，更大的历史保存在外部并按需 page in/out。 [22]

此时可以增加：

```
SummaryProvider
```

但必须是可选：

```
class HistorySummarizer(Protocol):
    async def summarize(
        self,
        groups: list[MessageGroup],
    ) -> HistorySummary:
        ...
```

结构化输出：

```
@dataclass(frozen=True)
class HistorySummary:
    task: str
    decisions: tuple[str, ...]
    completed_actions: tuple[str, ...]
    artifacts: tuple[str, ...]
    failures: tuple[str, ...]
    open_questions: tuple[str, ...]
    next_actions: tuple[str, ...]
```

在线 summary 可以按：

```
checkpoint boundary
而不是每 N messages
```

触发，例如：

```
exploration completed
root cause found
```

```
patch applied
tests passed
human approved
```

OpenAI 的当前部署指南也建议在“完成一个 meaningful milestone”后 compact，而不是等到最后一刻才处理窗口。[29]

长期也可以 provider-aware：

```
Anthropic
→ server-side compaction/context editing

OpenAI Responses
→ /responses/compact

Generic/local provider
→ local rule + summary
```

Anthropic 当前把 server-side compaction 作为主要长会话策略；OpenAI 则提供 standalone `/responses/compact` 和 server-managed compaction。[30]

不过 provider compaction应该放在 adapter 层：

```
HistoryCompactionPolicy
         ↓
Generic implementation
          │
          ├──── LocalRuleBased
          ├──── LocalHybrid
          ├──── AnthropicNative
          └──── OpenAINative
```

不要让：

```
LLMRunExecutor
```

直接知道：

```
Anthropic context_management JSON
```

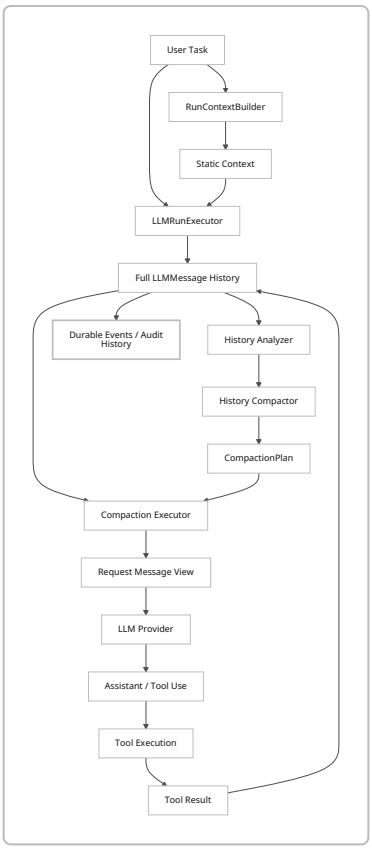
长期验收应加入“质量损失”而不仅是 token：

```
tokens/request
latency/request
run cost
```

```
tool-call count
tool-call correctness
artifact recall
task completion rate
summary factual consistency
recovery success
```

Mermaid 架构与集成图

当前消息流与 Compactor 正确集成点



这里最重要的是：

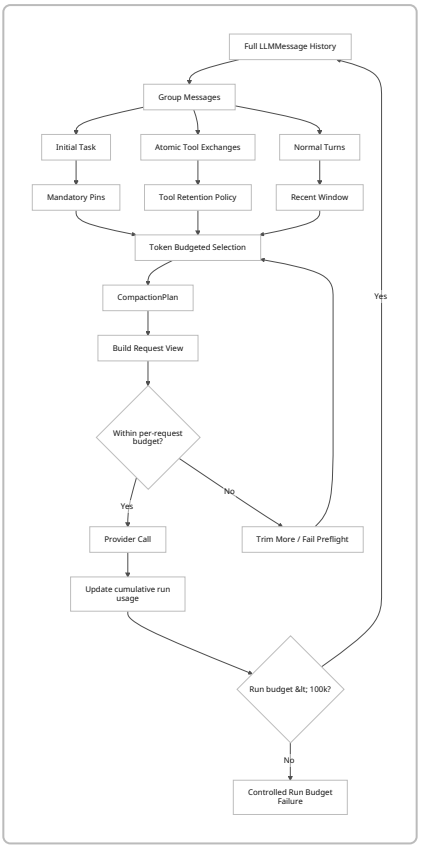
```
Compactor 不修改 Full History。
```

它只是产生：

```
本次 provider call 可见的 view。
```

这与 Anthropic context editing 中“客户端保留完整原 history、context editing 只改变模型本轮看到的 prompt”的思路一致。 12

推荐短期方案



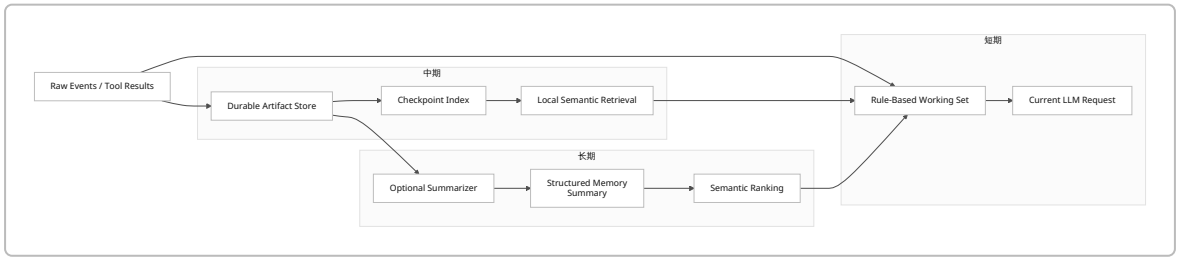
这个结构保留了你已经明确要求不能删除的：

```
100k cumulative run guard
```

同时新增了真正缺失的：

```
per-request history control
```

中期与长期差异



中期仍可做到：

```
0 次额外在线 LLM
```

长期才允许：

```
LLM summarization
+
semantic ranking
+
provider-native compaction
```

风险与注意事项

不要删除 system/instructions。 如果 system 独立于 `messages`，应从结构上让 compactor 永远碰不到它；如果未来 message model 支持 system role，则必须 permanent pin。Semantic Kernel reducer 的官方设计同样保留 system context。 [7]

不要按单 message 裁剪 tool interaction。 `tool_use` 和 `tool_result` 必须视为 protocol unit；LangChain 明确提醒多数 provider要求配对，Semantic Kernel 也专门有 threshold 机制以避免 orphan message。 [31]

Claude thinking + tool use 要更保守。 某些调用阶段要求把与 tool request 关联的 thinking block完整返回，签名内容不可修改；不能拿通用文本 compressor 任意裁剪这些 block。 [16]

Raw history 必须可回溯。 `CompactionPlan` 应作用于 request view，而不是 destructive edit 原数据。否则一旦 heuristic 判断错误，无法恢复。LangGraph checkpointing 的价值之一正是 fault tolerance 和 time travel。 [13]

所有压缩都要有审计日志。 至少记录：

```
original_messages
retained_messages
estimated_tokens_before
estimated_tokens_after
removed indices/groups
policy version
reason
```

Anthropic context editing 也会在 response 中报告具体 applied edits、cleared tool uses 和 cleared input tokens，这说明生产级 context management 本身就应具备 observability。 [12]

不要把 prompt caching 当 compaction。 caching 可以降低部分重复输入成本和延迟，但缓存内容仍然占上下文窗口；Anthropic 官方明确说明 cached prompt prefixes 仍占 window。 [32]

区分三类 token 计数。 至少要区分：

```
estimated context tokens
provider-reported request usage
cumulative run cost
```

provider 还有 cache tokens、reasoning tokens 等额外语义，不能用一个 `total_tokens` 变量表达所有概念。OpenAI reasoning token 与 Anthropic cache/input accounting 都说明了这一点。[9]

工具输出应该在源头减肥。 大量 JSON、目录树、日志、重复文件内容最好先 normalize/delta，而不是全部进入 history 后再删除。Semantic Kernel 官方就建议 tool return 仅包含模型需要的信息；Anthropic programmatic tool calling 可以让中间结果根本不进入模型 context。[20]

state-changing tool 和 read-only tool 风险不同。 `read_file` 的结果通常可重新取得；`apply_patch`、数据库写入、审批、git commit 等代表环境发生变化，应默认 pin 或建立 durable checkpoint。仅凭“旧”不能判定可删除。

测试结果应保留状态而非海量原日志。 最安全的模式通常是：

```
latest test summary
+
failed cases
+
artifact pointer to full log
```

而不是永久保留数千行 pytest/npm output。

并发与重入不能共享 mutable compaction state。 多 agent / parallel tool calls 时，每个 run/thread 应有独立的 history view、budget 与 plan；LangGraph 的 checkpointer 本身就以 thread-scoped state 组织会话。[33]

计划和执行要继续分离。 你现在已经有：

```
CompactionPlan
```

不要让 `RuleBasedHistoryCompactor` 自己 mutate messages。这样才可以分别测试：

```
policy correctness
plan correctness
apply correctness
runtime integration
```

序列化兼容性要测试。 `retained_indices` 依赖原 history 的 ordering 和 identity，因此 plan 最好只在同一 immutable snapshot 上 apply。未来如果支持 persistence，建议加：

```
history_revision / snapshot_id
```

避免拿旧 plan 应用到已增长的 message list。

不要让 embedding 打乱协议顺序。 LlamaIndex 官方明确指出 vector memory 不保证 message order；因此 semantic retrieval 的 index unit 必须是完整 exchange/artifact，而不能是 raw protocol message。[6]

Summarization 必须防止“事实升级”。 比如：

```
旧 history:
"可能是 race condition"

summary:
"root cause is race condition"
```

这种压缩错误会比 token overflow 更危险。长期 summary 应有显式字段：

```
confirmed facts
hypotheses
failed approaches
uncertainties
```

OpenAI 的可靠 agent compaction 示例也强调应保留 uncertainty，而不是在 memory/compaction 时偷偷把不确定性抹掉。[34]

应保留 human-in-the-loop 决策。 用户批准计划、拒绝某方案、限定不能修改某文件等都属于 high-priority context，不能因为 recency window 而消失。

错误诊断不能太早删除。 一个 tool error 应至少保留到：

```
对应问题被成功修复
+
有验证结果
```

否则 Agent 很可能再次重复同一个失败操作。

Compaction 必须可关闭和回滚。 推荐：

```
history_compaction_enabled: bool
```

以及：

```
policy_version
```

真实 smoke test 出现退化时可以立即切回 full-history baseline 做 A/B diagnosis。

不要仅测试 token 数。 长上下文越小并不必然等于任务质量越高。`Lost in the Middle` 说明位置和信息密度也会影响利用率，所以 eval 必须同时测 task correctness、tool correctness 与关键事实 recall。[35]

run-level cumulative guard 必须继续存在。 History compaction 的目标不是允许 agent 永久循环。即使每轮只有 2k token，一个失控的 100 轮 tool loop 仍应被成本/步骤护栏终止。LangChain 当前也把 model-call

limit 和 tool-call limit列为 agent middleware 的独立安全能力，而不是用 context compaction 替代它们。[36]

结论与推荐

现有框架和研究给出的方向非常一致：

```
最差：
Full History Forever

稍好：
Recent N Messages

更好：
Head + Tail

Agent 更合适：
Head
+
Atomic Recent Tool Exchanges
+
Token Budget

生产级：
上述方案
+
Tool Pinning
+
Durable Checkpoints
+
Artifact Retrieval

长期：
上述方案
+
Semantic Retrieval
+
Optional Summarization
```

AutoGen 的 recent-N/token-limit/head-tail 三套 context implementation、LangChain 的 trim-before-model、Semantic Kernel 的 reducer abstraction、LlamaIndex 的 token-budgeted short-/long-term memory，以及 Anthropic 的 tool-result clearing，本质上都支持这一判断。[37]

因此，**我不建议现在按此前设计直接落一个固定 `RECENT_MESSAGE_COUNT = 6` 的 `RuleBasedHistoryCompactor`**。研究之后，应该把 **5D-6B-4B-2** 的目标稍微升级，但不扩大到 LLM summarization：

```
5D-6B-4B-2
Tool-Aware RuleBasedHistoryCompactor
```

```
MessageGroupBuilder
        ↓
Atomic Tool Exchanges
        ↓
Mandatory Pins
        ↓
Recent Sliding Window
        ↓
Token Budget
        ↓
CompactionPlan
```

具体推荐优先级是：

```
立即做
──────────────────────────────────────────────

1. Atomic tool exchange grouping
2. First-task / mandatory pinning
3. Token-budgeted newest-first retention
4. RuleBasedHistoryCompactor
5. CompactionPlan apply layer
6. LLMRunExecutor before-provider integration
7. full-history audit preservation
8. real smoke test

随后做
──────────────────────────────────────────────

9. ToolResultRetentionPolicy
10. delta / succinct tool output
11. checkpoints + artifact pointers

暂缓
──────────────────────────────────────────────

12. keyword/heuristic scoring refinement
13. local embedding retrieval
14. LLM summarization
15. provider-native compaction
16. learned prompt compression
```

这个顺序最符合你当前已经建立的架构原则：**先纯数据模型，再规则策略，再执行层，最后 runtime integration；保持 deterministic，可测试、可解释、可回滚。**

同时它不会打乱原来的项目主线。阶段地图应保持为：

```
5D-5 Planning + Human Governance
✅

5D-6A CLI Human Loop
✅

5D-6B Runtime Context Fix

    Static Context
     ├──── Context Model
     │    ✅
     ├──── Context Selector
     │    ✅
     ├──── Context Budget
     │    ✅
     ├──── Context Estimator
     │    ✅
     ├──── Context Trimmer
     │    ✅
     └──── Runtime Integration
          ✅

    Dynamic History
     ├──── History Analyzer
     │    ✅
     ├──── CompactionPlan
     │    ✅
     ├──── Tool-Aware RuleBased Compactor
     │    ← NEXT
     ├──── Compaction Executor
     │
     └──── LLMRunExecutor Integration


之后恢复原路线:

RetryManager
    ↓
Approval Persistence
    ↓
Claude Planner / advanced planning
```

最核心的架构判断可以浓缩成一句：

> **对当前项目，不应“删除历史来省 token”，而应“永久保存完整历史，只给 LLM 一个按 protocol group、重要性、时间与 token budget 构造出来的有限 working set”。**

这既是短期最低风险的修复，也是未来 checkpoint、RAG memory、summarization 和 provider-native compaction 都能继续向上演进的正确基础。 [38]

---

1 2 5 11 15 23 27 31 Short-term memory - Docs by LangChain
https://docs.langchain.com/oss/python/langchain/short-term-memory

3 22 38 MemGPT: Towards LLMs as Operating Systems
https://arxiv.org/abs/2310.08560?utm_source=chatgpt.com

4 16 Context windows - Claude Platform Docs
https://docs.anthropic.com/en/docs/build-with-claude/context-windows

6 19 Memory | Developer Documentation
https://developers.llamaindex.ai/python/framework/module_guides/deploying/agents/memory/

7 Creating and managing a chat history object | Microsoft Learn
https://learn.microsoft.com/en-us/semantic-kernel/concepts/ai-services/chat-completion/chat-history

8 35 Lost in the Middle: How Language Models Use Long Contexts
https://arxiv.org/abs/2307.03172?utm_source=chatgpt.com

9 Reasoning models | OpenAI API
https://developers.openai.com/api/docs/guides/reasoning?utm_source=chatgpt.com

10 14 37 autogen_core.model_context — AutoGen
https://microsoft.github.io/autogen/stable//reference/python/autogen_core.model_context.html

12 Context editing - Claude Platform Docs
https://platform.claude.com/docs/en/build-with-claude/context-editing

13 21 28 33 Persistence - Docs by LangChain
https://docs.langchain.com/oss/python/langgraph/persistence?utm_source=chatgpt.com

17 semantic_kernel.contents.history_reducer.chat_history_truncation_reducer module | Microsoft Learn
https://learn.microsoft.com/en-us/python/api/semantic-kernel/semantic_kernel.contents.history_reducer.chat_history_truncation_reducer?view=semantic-kernel-python&utm_source=chatgpt.com

18 Token counting - Claude Platform Docs
https://docs.anthropic.com/en/docs/build-with-claude/token-counting?utm_source=chatgpt.com

20 Function calling with chat completion | Microsoft Learn
https://learn.microsoft.com/en-us/semantic-kernel/concepts/ai-services/chat-completion/function-calling/?utm_source=chatgpt.com

24 Compaction | OpenAI API
https://developers.openai.com/api/docs/guides/compaction?utm_source=chatgpt.com

25 Memory and RAG — AutoGen
https://microsoft.github.io/autogen/stable//user-guide/agentchat-user-guide/memory.html?utm_source=chatgpt.com

26 LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression
https://arxiv.org/abs/2310.06839?utm_source=chatgpt.com

29 API deployment checklist
https://developers.openai.com/api/docs/guides/deployment-checklist?utm_source=chatgpt.com

30 Context windows - Claude Platform Docs
https://docs.anthropic.com/en/docs/build-with-claude/context-windows?utm_source=chatgpt.com

32 Prompt caching - Claude Platform Docs
https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching?utm_source=chatgpt.com

34 Building Reliable Agents with Memory and Compaction
https://developers.openai.com/cookbook/examples/agents_sdk/building_reliable_agents_memory_compaction?utm_source=chatgpt.com

36 Prebuilt middleware - Docs by LangChain
https://docs.langchain.com/oss/python/langchain/middleware/built-in?utm_source=chatgpt.com